

How Do Community Smells Influence Self-Admitted Technical Debt in Machine Learning Projects?

Shamse Tasnim Cynthia Nuri Almarimi Banani Roy
Department of Computer Science, University of Saskatchewan, Canada
{shamse.cynthia, nuri.almarimi banani.roy}@usask.ca

Abstract—Community smells reflect poor organizational practices that often lead to socio-technical issues and the accumulation of Self-Admitted Technical Debt (SATD). While prior studies have explored these problems in general software systems, their interplay in machine learning (ML)-based projects remains largely underexamined. In this study, we investigated the prevalence of community smells and their relationship with SATD in open-source ML projects, analyzing data at the release level. First, we examined the prevalence of ten community smell types across the releases of 155 ML-based systems and found that community smells are widespread, exhibiting distinct distribution patterns across small, medium, and large projects. Second, we detected SATD at the release level and applied statistical analysis to examine its correlation with community smells. Our results showed that certain smells, such as Radio Silence and Organizational Silos, are strongly correlated with higher SATD occurrences. Third, we considered the six identified types of SATD to determine which community smells are most associated with each debt category. Our analysis revealed authority- and communication-related smells often co-occur with persistent code and design debt. Finally, we analyzed how the community smells and SATD evolve over the releases, uncovering project size-dependent trends and shared trajectories. Our findings emphasize the importance of early detection and mitigation of socio-technical issues to maintain the long-term quality and sustainability of ML-based systems.

Index Terms—community smell, self-admitted technical debt, machine learning.

I. INTRODUCTION

Community smells arise from poor organizational and social practices that contribute to the buildup of social debt [1]–[3]. Social debt leads to communication breakdowns, weak collaboration, and coordination issues [2], [4]. Such socio-organizational deficiencies are known as “community smells”. If left unaddressed, these sub-optimal patterns can increase project complexity and cost, ultimately leading to the accumulation of technical debt and impacting software quality [5]. These challenges are equally relevant in machine learning (ML) projects where the human and collaborative aspects of development remain underexplored. As a result, teams often face difficulties in coordinating work, defining interfaces, and managing system evolution [6]. Nahar et al. [6] identified factors such as cultural differences, power struggles, and uneven expertise as key contributors to

collaboration breakdowns in ML-based systems. These socio-technical issues can, in turn, lead developers to take shortcuts, apply quick fixes, or defer proper solutions, which they often document explicitly in the form of Self-Admitted Technical Debt (SATD) [7], [8]. Disputes, discomfort, or conflicts among developers due to socio-technical factors can significantly contribute to the emergence of SATDs during project development. The more technical debt persists, the more expensive it becomes, demanding substantial time, effort, and knowledge from developers to resolve. Therefore, to better understand the interplay between organizational dynamics and SATD in ML projects, this study aims to: (1) examine the prevalence of community smells in ML-based systems, (2) explore their co-occurrence with SATD, (3) identify which SATD types are most associated with specific community smells, and (4) analyze how both evolve across releases in projects of different sizes.

Prior studies have explored the relationship between community smells and various technical and social aspects of software development [7], [13]–[17]. For instance, Palomba et al. [13] linked code and community smells, emphasizing the need for active communication among developers to enhance maintainability and reduce technical debt. Tamburri et al. [15] found that community smells often align with sub-optimal architectural structures, while Mumtaz et al. [16] explored their relationship with design smells. On the other hand, in the context of ML systems, Mailach et al. [17] identified leadership gaps, organizational silos, and poor communication as key socio-organizational challenges, and Zampetti et al. [7] showed that organizational and project cultures significantly influence developers in introducing SATD [8]. Although Annunziata et al. [18] have explored community smells in ML projects, they typically focused on existing literature and extended it through developer interviews and supporting statistical models. Additionally, no studies have explored the prevalence of community smells in the ML systems at the release level. While previous work highlights socio-technical challenges in ML development, the interplay between community smells and SATD over project evolution remains underexplored.

In this study, we addressed these gaps through an empirical investigation of 155 open-source ML projects, analyzing them at the release level. We aimed to detect *ten* types of community

TABLE I: Details of Community Smells

Name	Definition
Organizational Silo Effect (OSE)	This smell refers to distinct subgroups within a community and poor communication among developers, which leads to wasted time, duplicated code, and added project costs. [1], [4], [9].
Black-cloud Effect (BCE)	This smell reflects information overload due to poor communication, limited knowledge-sharing opportunities, and a lack of experienced members to bridge gaps in expertise. [1], [4].
Prima-donnas Effect (PDE)	This smell originates when a group of individuals is resistant toward change proposed by other team members due to poorly organized collaboration. [1], [4], [9].
Sharing Valley (SV)	This smell stems from a lack of effective information exchange, such as face-to-face meetings, that leads to the spread of outdated or unreliable knowledge. [1].
Organizational Skirmish (OS)	This smell arises from mismatched expertise and communication gaps among team members, often leading to lower productivity and affecting the project's timeline and budget. [1].
Solution Defiance (SD)	This smell arises from cultural and experiential differences, leading to opposing subgroups with conflicting views on technical and socio-technical decisions. The presence of this smell often results in project delays and uncooperative behavior. [1].
Radio Silence (RS)	This smell originates from excessive formality and poor organizational structure, slowing down changes and wasting time. It often causes significant delays in decision-making due to the rigidity of formal procedures [1].
Truck Factor Smell (TFS)	The smell occurs when critical knowledge is concentrated in a few developers, leading to significant knowledge loss if they leave the project. [10].
Unhealthy Interaction (UI)	This smell arises when slow, shallow or low-quality discussions, with minimal developer engagement and long delays, take place between message exchanges. [11].
Toxic Communication (TC)	This smell is present when developer communications involve harmful dialogues and express negative emotions, including dissatisfaction, anger, or discordant viewpoints on different issues under discussion. Such interaction can cause frustration and may lead developers to disengage from the project. [11], [12].

smells and their prevalence in the ML projects. Additionally, we conducted a correlation analysis to explore the relationship between community smells and SATD, and further examined which types of community smells are most associated with specific *six* SATD categories. Furthermore, we analyzed how both community smells and SATD evolve across project releases in ML systems of varying sizes. In particular, this study answered *four* research questions and made *four* key contributions, as outlined below:

RQ1: To what extent is the prevalence of community smells in ML-based systems?

We examine the prevalence of various community smells in ML projects to understand how socio-technical issues manifest in these systems. This analysis is essential for identifying early collaboration bottlenecks, guiding organizational improvements, and fostering healthier ML development practices.

RQ2: To what extent do community smells co-occur with SATD in ML-based systems?

We investigate the relationship between community smells and the presence of SATD in ML projects. Understanding this co-occurrence helps uncover how socio-technical issues contribute to technical debt accumulation and informs targeted strategies for mitigating both.

RQ3: What types of SATD are most associated with specific community smells?

We analyze the associations between individual community smells and distinct types of SATD to identify which socio-technical issues most strongly contribute to specific forms of technical debt. This insight supports more focused interventions to address the root causes of recurring debt patterns in ML-based systems.

RQ4: How do community smells and SATD evolve over releases in ML projects of different sizes? We examine the temporal trends of community smells and SATD across small, medium, and large ML projects to understand how these

issues progress throughout a project's lifecycle. This analysis highlights size-specific patterns and informs proactive debt management and collaboration strategies over time.

Replication Package is available in our online appendix [19].

II. BACKGROUND

1) *Community Smells*: “Community smell” is a term within an advanced research field that delineates socio-technical characteristics (e.g., high formality) and patterns (e.g., repeated condescending behaviour, rage quitting) leading to the emergence of social debt [20]. Previous studies have identified a range of community smells that reflect socio-technical issues within development teams. In our research, we refer to the ten types of community smells detected by the csDetector tool [21]. Table I summarizes the definition of ten different types of community smells studied in our research.

2) *Self-admitted Technical Debt*: Ward Cunningham initially introduced the concept of technical debt in 1993 as a metaphor, highlighting the inevitable costs, such as maintenance and evolution expenses, that developers incur when dealing with code that is not quite optimal [22]. Technical debt often arises due to tight deadlines, high-quality demands, or the need for rapid delivery, which compel developers to make short-term trade-offs [23]–[26]. When developers are aware that their short-term implementations are suboptimal, they may signify this by adding source code comments, a phenomenon recognized as SATD [8]. Prior research by Wehaibi et al. [27] indicates that the presence of SATD in certain projects undergoes more bug-fixing changes, making these debt-related modifications more challenging.

III. RELATED WORK

Community smells in software repositories: The influence of community smells on various aspects of software development has been extensively studied [3], [28]–[31]. Palomba et al. [28] found that developers often perceive a

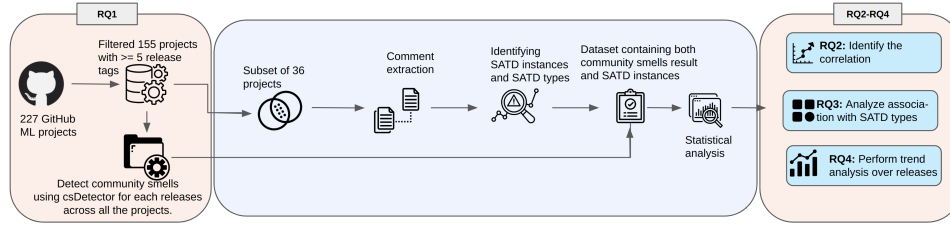


Fig. 1: Schematic diagram of our methodology

connection between code and community smells. Sarmento et al. [29] examined the role of gender diversity and refactoring in mitigating these smells. Caballero-Espinosa et al. [30] framed community smells as social debt and reviewed their implications. Almarimi et al. [3], [31] proposed machine learning-based detection models that leverage socio-organizational patterns and later extended this to multi-label detection of eight smell types [32]. Similarly, Voria et al. [33] introduced CADOCS, a conversational agent built on community smell detection tools. However, in the context of ML-based systems, community smells have not been studied comprehensively. A recent study by Annunziata et al. [18] tried to identify the types, causes, effects and potential mitigation strategies of community smells in ML-enabled systems. However, their work is built on existing research by extending it through interviews and reinforcing it with statistical models. Conversely, Nahar et al. [6] highlighted that human collaboration factors are often overlooked in ML projects, despite posing significant challenges. Through interviews with 45 developers, they examined issues related to team structure, cross-role interaction, and conflict across platforms. Moreover, Mailach et al. [17] identified 17 non-technical anti-patterns tied to poor communication and organizational gaps. Other studies confirm that interdisciplinary ML teams often face collaboration barriers due to cultural, methodological, and expectation mismatches [34]–[36]. Given the clear impact of poor organizational and social practices on ML-based development, our study investigates the prevalence of community smells in ML systems, focusing on the specific smell types that have not been explored in this context before.

SATD in different sources: Sierra et al. [37] have highlighted that developers acknowledge the presence of SATD from various sources. Zampetti et al. [7] surveyed 101 developers and found that source code comments are the most common method for documenting SATD in both industrial and open-source projects. Additionally, several empirical studies have investigated SATD. Potdar and Shihab [8] analyzed source code comments from four large open-source projects and found that SATD prevalence ranged from 2.4% to 31% across files. Their study also revealed that more experienced developers were responsible for introducing most of the SATD. Moreover, a wide range of research has been done to identify SATD in the software repositories over the years [38]–[40]. Researchers have also explored SATD in ML-based systems. O’Brien et al. [41] conducted a large-scale study on technical debt in ML software, analyzing 68,820 SATD instances across

2,641 curated GitHub repositories. Their findings revealed five new and five split SATD types specific to ML projects. Not only that, Bhatia et al. [42] further found that ML projects have a median SATD percentage that is twice as high as that of non-ML projects. However, to the best of our knowledge, no prior study has thoroughly investigated the reasons behind the high prevalence of SATD in ML-based projects. Given that community smells often signal underlying technical and organizational issues, our study aims to explore whether a correlation exists between community smells and SATD instances and their evolution over the time.

IV. METHODOLOGY

This section describes our study research approach to answer the research questions. Fig. 1 shows the schematic diagram of our study methodology.

A. Dataset Description

The first step of our study involved selecting appropriate ML projects. To this end, we built upon the work of Latendresse et al. [43] for two primary reasons. First, their contribution provides a curated dataset of 227 open-source ML projects, offering a robust foundation for our analysis. These projects were carefully selected to represent active, high-quality, and diverse repositories. Second, their selection criteria ensured that the chosen projects are mature and stable in their development lifecycle characteristics. These criteria are crucial for our investigation, as mature projects are more likely to have experienced and responded to various challenges, thereby offering a richer and more meaningful dataset to investigate the prevalence of community smells and their relationship with SATD. Given our focus on community-related smells, this dataset aligns well with the objectives of our study.

To investigate the relationship between community smells and SATD, we needed to observe both within the same period of project development, i.e., they happen at the same time or close to each other. For this reason, we chose to analyze the presence of both patterns at the release level [28]. We filtered the original 227 projects to keep only those with at least five tagged releases to ensure a richer dataset with enough

Type	Community Dimension	# of Projects	# of releases
Small	< 50	81	3081
Medium	50 – 150	44	1867
Large	> 150	30	1521

information for meaningful analysis. After applying this filter, we selected 155 projects for our study. To observe the prevalence of community smells based on the community size, we categorized the projects into small, medium, and large

TABLE II: Breakdown of the projects

projects following Tamburri et al. [44]. Table II shows the breakdown of the project types.

B. Analyzing the Prevalence of Community Smells

In order to detect community smells for every release of each project, we utilized the csDetector tool [46]. This sophisticated tool utilizes an ML-based detection method that provides insights from numerous instances of suboptimal community development practices, facilitating automated identification of similar issues within the community. The csDetector tool is well-known in the software engineering community [31], [47]–[50]. There are some other tools mentioned in the existing studies [4], [51]–[53]. However, in our study, we use csDetector for several key reasons: 1) the tool offers a comprehensive framework for detecting community smells by integrating diverse data sources, including social network graph analysis (covering commits, pull requests, and issue report discussions), sentiment analysis, and truck factor metrics, 2) csDetector exhibits high performance, achieving an average F1 score of 84% across different types of community smells, based on an analysis of 143 open-source projects and 3) csDetector can detect ten types of community smells and it detects the smells based on 52 socio-technical metrics whereas the other state-of-the-art tools can detect four types of community smells [4] or identify development communities based on six factors [51].

Next, we introduced two parameters in the csDetector tool, *start_date* and *end_date*, to enable release-based analysis of the repositories. For each project, the tool iterates through the repository from the date of one release to the next, detecting community smells within that specific time frame. Since csDetector relies on ML-based detection methods, the results may vary slightly across runs due to inherent non-determinism in the model [54]. To ensure the reliability of the outputs, we conducted a robustness check: for a randomly selected sample of 10 projects, we executed the tool five times per release and manually examined the detected smells. The first two authors of this study independently reviewed the outputs. To improve transparency and address concerns about the depth of our manual analysis, we developed a rubric based on prior literature to guide our classification and ensure structured, repeatable assessment. In the event of a conflict regarding the presence of a community smell, where there is disagreement about its existence, the authors seek to resolve it through open discussion. This approach aims to reach a consensus on the key symptoms of the potential smell. Our validation process revealed that in one out of the ten projects, the detection of the Toxic Communication (TC) smell did not reach full agreement for some releases. Consequently, we excluded that smell from the release’s detected smells and re-evaluated the detection of this specific smell across all other projects. Ultimately, the authors achieved full agreement with the tool’s results, ensuring the reliability and accuracy of the community smell detections in our study. Next, following the methodology of Tamburri et al. [4], we conducted a quartile analysis to

examine the distribution and prevalence of community smells across ML-based projects.

C. Investigating the Correlations between Community Smells and SATD

We investigated the correlation between community smells and SATD occurrences in ML projects using Spearman’s rank correlation [55], as the data was not normally distributed [56]. Spearman’s correlation measures the strength and direction of monotonic relationships between variables. To detect a moderate effect size ($\rho = 0.3$) with $\alpha = 0.05$ and power = 0.8, a minimum of 82 observations is typically required [57]. We selected a stratified sample of 36 projects (19 small, 10 medium, 7 large), preserving the original size distribution of the dataset (52.6% small, 28.2% medium, 19.2% large) [58]. These projects collectively contain over 800 releases, offering more than enough data points to meet the power requirements and support reliable correlation analysis. After identifying the sampled projects, we proceeded to extract SATD-related comments to analyze their relationship with community smells. We collected source code comments from each project’s release history by cloning their GitHub repositories and checking out each tagged release. We targeted source files with common extensions (.py, .java, .js, .cpp, .c), using predefined patterns to extract single-line comments. These comments were aggregated per release and saved in structured JSON files.

To identify SATD instances within the extracted comments, we employed the MT-BERT-SATD model proposed by Gu et al. [59], which is based on multi-task learning using BERT. The model achieved an average F1-score of 0.712 (ranging from 0.625 to 0.859), outperforming previous approaches by 4.6% to 30.1%. The model has demonstrated effective generalization across programming languages and software artifacts. We applied this model to the collected comment sets and stored the predictions for each project and release. For every release, we recorded both the number of detected SATD instances and the number of community smells. The complete results are available in the online appendix [19]. Finally, we applied Spearman’s rank correlation independently for each project to examine whether a statistically significant correlation exists between these two factors.

D. Exploring the association between SATD types and community smells

To further examine the relationship between community smells and SATD, we analyzed which types of SATD are more closely associated with specific community smells. Therefore, we classified the detected SATD comments using a Large Language Model (LLM). Sheikhaei et al. [60] evaluated the effectiveness of LLMs in both identifying and classifying SATD comments, and found that an ensemble approach based on the Flan-T5-XL¹ model outperformed other models in terms of classification accuracy. Following their methodology,

¹<https://huggingface.co/google/flan-t5-xl>

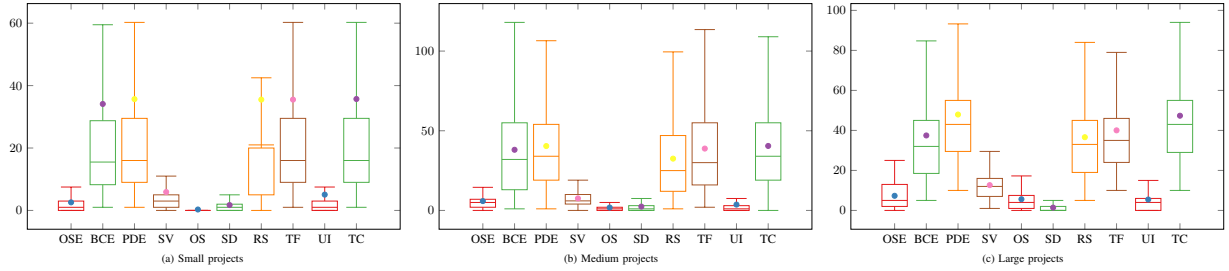


Fig. 2: Quartile analysis of community smells by project size (● represents the mean value).

TABLE III: Quartile statistics of SATD-related community smell occurrences by project size

Size	Metric	OSE	BCE	PDE	SV	OS	SD	RS	TF	UI	TC
Small	Median	1.00	15.50	16.00	3.00	0.00	1.00	11.00	16.00	1.00	16.00
	Average	2.59	34.11	35.66	5.90	0.30	1.72	26.98	35.51	5.07	35.70
	Min	0.00	1.00	1.00	0.00	0.00	0.00	0.00	1.00	0.00	1.00
	Max	7.50	59.50	60.25	11.00	0.00	5.00	42.50	60.25	7.50	60.25
	Std. Dev	6.05	104.18	110.54	15.64	1.29	5.13	102.79	110.56	24.75	110.53
Medium	Median	5.00	32.00	35.50	6.00	1.00	1.00	25.50	30.50	1.00	35.50
	Average	5.85	38.06	40.40	7.54	1.81	2.41	32.51	38.82	3.66	40.47
	Min	0.00	1.00	1.00	0.00	0.00	0.00	1.00	2.00	0.00	0.00
	Max	14.50	116.63	105.88	18.38	4.59	7.50	96.50	112.25	7.80	108.38
	Std. Dev	6.50	28.76	28.09	6.62	3.98	4.56	26.15	28.59	6.58	28.11
Large	Median	5.50	31.50	42.50	12.00	4.00	0.50	32.00	34.00	4.00	42.50
	Average	7.43	34.63	45.33	12.90	5.77	1.47	33.83	37.27	5.63	44.73
	Min	0.00	5.00	10.00	1.00	0.00	0.00	5.00	10.00	0.00	10.00
	Max	25.00	80.25	93.63	29.13	17.88	5.00	75.25	75.25	14.63	94.00
	Std. Dev	6.45	23.16	25.05	8.28	6.37	2.70	22.89	23.90	7.93	25.73

TABLE IV: # of ML projects affected by community smells

Project type	OSE	BCE	PDE	SV	OS	SD	RS	TF	UI	TC
Small	53	81	81	69	11	46	80	81	44	81
Medium	39	44	44	42	23	29	44	44	26	43
Large	25	30	30	30	25	14	30	30	21	30
Overall	117	155	155	141	59	89	154	155	91	154

TABLE V: Definitions and examples of six SATD types [45]

SATD Types	Descriptions	Example Comment
Requirement	Requirement debt can either be functional (incomplete features) or non-functional (falls performance, memory, or security standards).	TODO: handle channel modalities later
Code	Poor coding practices reduce code legibility, making it hard to understand and maintain.	TODO: This next code is dense and confusing. Clean up at some point.
Monitoring and Testing (M&T)	Issues in implementing testing or monitoring components.	XXX: should we rather test if instance of estimator?
Defect	Detected system defects that need resolution	TODO this will fail if a parameter cant handle size=(N:)
Design	Violations of good design practices that hinder adaptability to changing business needs	TODO maybe improve this so it doesn't use a global.
Documentation	Insufficient documentation within the software system.	TODO update doc above.

we used the same model and settings to categorize SATD instances in our dataset. In terms of SATD taxonomy, prior work by O'Brien et al. [45], introduced *six* general categories of SATD, i.e., Requirement, Code, M&T, Defect, Design, and Documentation, as well as 23 domain-specific subcategories for ML-related projects. However, since Sheikhaei et al.'s [60] work focused on general SATD classification and we aimed to explore the broader associations across project types, we adopted only the six general SATD categories in our study. Table V summarizes the description of six types of SATD. The classified SATD types were stored alongside the community smell detection results for each project release. Full results are available in the online appendix [19].

Next, to analyze the association between specific SATD types and community smells, we applied the Point-Biserial

Correlation test [61], [62]. This statistical method is appropriate when one variable is binary and the other is continuous. In our case, the presence or absence of each of the ten community smells types serves as the binary variable, while the number of instances for each of the six SATD types serves as the continuous variable. This allowed us to assess the strength of association between individual SATD types and specific community smells across project releases.

E. Analyzing trends of community smells and SATD over releases

We employed the Mann-Kendall test [63] to assess whether community smells and SATD instances exhibit upward, downward, or no trends over project releases. The null hypothesis assumes no trend, while the alternative suggests a monotonic trend. The test's coefficient indicates the direction and strength of the trend. By applying this test to each project, we examined whether community smells and SATD evolve in parallel over time. A consistent trend in both metrics may suggest a potential relationship between them.

V. FINDINGS

A. Answering RQ1: Prevalence of community smells in ML projects

Fig. 2 depicts the box plots reporting the distribution of the community smells analyzed over the 155 ML projects in our dataset. These plots reflect the overall distributions by aggregating community smell occurrences over all releases in the three categories. Next, Table III reports the median, average, minimum, maximum, and standard deviation observed in each of the releases for all the projects. As shown in Fig. 2, five community smells, Black Cloud

Effect (BCE), Prima-donna Effect (PDE), Radio Silence (RS), Truck Factor Smell (TFS), and Toxic Communication (TC), are the most prevalent across all project sizes, with consistently high median and mean values. In small projects, the average occurrences of BCE, PDE, TFS, and TC range from 34-36, though high standard deviations (above 100) suggest that some teams face severe socio-technical issues despite lower overall averages. However, these smells intensified in medium projects, with PDE and TC reaching median values of 35.5, BCE at 32.0, and TFS at 30.5. RS also showed increased presence, with a median of 25.5. Interestingly, large projects followed a similar trend, exhibiting medians of 42.5 for both PDE and TC, 31.5 for BCE, 34.0 for TFS, and 32.0 for RS. The higher differences of PDE and TC in large projects compared to small projects suggest that smells related to knowledge centralization and hostile communication become significantly more prominent as projects scale. Furthermore, Sharing Villainy (SV) and Organizational Silo Effect (OSE) show a steady increase in median values from small to large projects, with SV rising from 3.0 to 12.0 and OSE from 1.0 to 5.5. This reflects a trend where growing team size and complexity lead to less efficient knowledge sharing and more isolated collaboration. In contrast, Organizational Silo Effect (OSE), Solution Deficiency (SD), and Unhealthy Interaction (UI) were less prevalent overall but displayed unique distribution patterns worth further investigation.

Next, from Table IV, we observed that the five most frequent community smells are present in almost all of the projects analyzed. Each of these smells was detected in at least 154 out of 155 projects. This widespread presence indicates that the occurrence of these smells is not solely dependent on community size. However, the pattern is notably different for the Organizational Skirmish (OS) smell. Only 11 projects with fewer than 50 contributors exhibited OS, and the number of affected projects gradually increases with larger community sizes. This trend suggests that the presence of OS may be influenced by the size of the community, indicating a potential correlation between contributor count and OS occurrences.

Summary for RQ₁: Community smells are highly prevalent in ML projects but not uniformly distributed. BCE, PDE, RS, TFS and TC are found regularly, while the presence of OS instances may depend on the number of contributors within a community.

B. Answering RQ₂: Correlation Between Community Smells and SATD Types

The previous section showed the prevalence of community smells in ML projects. This section analyzes whether the community smells help in the emergence of SATD instances, i.e., whether community smells have any correlation with SATD instances. As shown in Table VI, we found significant correlations with SATD in 8 out of 19 cases among the small projects; for medium projects, 8 out of 10 exhibited significant relationships; and in large projects, 6 out of 7

TABLE VI: Spearman Rank Correlation Between Community Smells and SATD Per Project

Community dimension	ID	Project	ρ	p -value	Significance
small	P1	bigartm	0.73	0.0002	***
	P2	coco-annotator	0.53	0.005	**
	P3	PyKEEN	0.57	0.009	**
	P4	POT	0.68	0.0000	***
	P5	OpenVINO-model-server	0.44	0.03	*
	P6	dm-control	0.47	0.01	*
	P7	EconML	0.41	0.02	*
	P8	interpret	0.33	0.02	*
medium	P9	pyro	0.71	0.0000	***
	P10	ludwig	0.55	0.0000	***
	P11	hdbscan	0.44	0.0008	***
	P12	compromise	0.44	0.0000	***
	P13	nlp.js	0.34	0.02	*
	P14	scanpy	0.27	0.01	*
	P15	MMdnn	0.70	0.02	*
	P16	deepchem	0.58	0.009	**
large	P17	tesseract	0.48	0.006	**
	P18	incubator-mxnet	0.55	0.0015	**
	P19	shap	0.28	0.04	*
	P20	mne-python	0.28	0.04	*
	P21	OpenNMT-py	0.32	0.03	*
	P22	kornia	0.33	0.04	*

Note: * $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$

demonstrated significant associations. Overall, our Spearman rank correlation analysis indicates that releases with a higher number of community smells tend to exhibit an increased number of SATD instances. Based on this finding, we further analyzed the relationship between individual community smells and the number of SATD instances per release across 23 ML projects. Table VII summarizes the overall result. As shown in the Table, Radio Silence (RS) emerges as a consistently significant factor. Small projects such as *P2* and *P4*, medium projects such as *P10* and *P12*, and large projects such as *P17* and *P20* all exhibit strong, positive correlations between RS and SATD. These findings suggest that procedural bottlenecks and delayed communication may contribute to the accumulation of SATD as necessary changes are de-prioritized in overly formal development environments. Sharing Villainy (SV) consistently correlates with SATD across small (e.g., *P3*, *P7*), medium (*P9*, *P14*), and large (*P19*) projects, suggesting that outdated or overly specialized solutions contribute to technical debt. In fast-evolving ML systems, such knowledge gaps delay debt resolution for the entire team. ML systems evolve rapidly, so gaps or outdated knowledge slow down technical debt fixes for everyone. Organizational Silo Effect (OSE) consistently correlates with SATD across project sizes, small (*P1*, *P4*), medium (*P9*, *P11*), and large (*P17*, *P20*). Silos fragment collaboration hindering coordinated debt management. As teams grow, OSE's prevalence rises, further reinforcing the idea that fractured group structures become more detrimental with increasing project size. In ML projects such silos may prevent timely resolution of design or implementation trade-offs, thereby contributing to sustained or growing SATD levels. Together, RS, SV, and OSE emerge as key socio-technical challenges tied to SATD in ML projects.

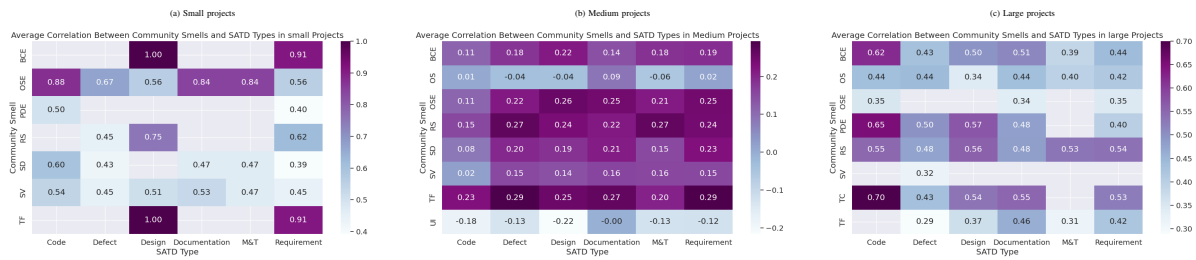
However, despite overall similarities, each project size shows unique patterns between community smells and SATD. Small projects often have few or no significant correlations, but when present, the effects can be strong. For example, *P4* shows strong associations with OSE, SV and RS, whereas *P5* and *P6* lack enough data for meaningful analysis. Interestingly, medium projects reveal more complex and sometimes

TABLE VII: Spearman Rank Correlation Between Individual Community Smells and SATD Per Project

	Project	OSE	BCE	PDE	SV	OS	SD	RS	TF	UI	TC
Small	P1	0.613***	0.368	—	0.346	—	0.354	0.368	0.368	0.198	—
	P2	0.448*	—	—	0.358	—	0.358	0.456*	-0.248	—	—
	P3	0.352	0.02	—	0.566**	—	—	—	—	—	—
	P4	0.615***	-0.129	—	0.501**	—	0.341	0.516**	—	-0.033	—
	P5	—	0.122	—	0.137	—	—	0.286	0.274	0.187	—
	P6	-0.085	—	0.451*	0.17	—	—	—	—	—	—
	P7	0.205	—	—	0.481**	—	—	-0.092	—	—	—
	P8	0.025	-0.05	—	0.216	—	—	0.174	—	0.166	—
Medium	P9	0.481**	0.287	—	0.446**	—	0.151	0.275	0.356*	—	—
	P10	0.011	0.401**	—	-0.315*	-0.38**	—	0.498***	0.353*	-0.194	—
	P11	0.297*	—	—	0.446***	—	0.322*	0.316*	-0.148	—	—
	P12	0.123	-0.192	—	0.262*	0.108	0.137	0.383***	-0.108	0.175	—
	P13	0.116	-0.116	—	0.167	—	0.116	0.296	—	0.116	—
	P14	0.201	—	—	0.35***	—	—	-0.118	—	—	—
	P15	0.574	0.43	—	0.574	0.0	0.382	0.344	0.462	-0.205	—
	P16	—	-0.094	—	0.02	0.25	0.391	0.166	0.311	0.011	—
Large	P17	0.402*	0.565***	0.604***	-0.224	—	-0.455**	0.689***	0.131	-0.578***	0.616***
	P18	0.148	0.417*	—	0.008	0.387*	—	0.257	0.322	0.017	0.291
	P19	0.107	-0.05	—	0.282*	-0.053	0.166	0.177	-0.307*	0.109	—
	P20	0.387**	0.219	—	-0.278*	0.019	—	0.426***	0.16	-0.538***	—
	P21	0.126	0.25	—	0.161	-0.25	-0.036	0.071	0.283*	—	—
	P22	0.316*	-0.08	—	0.168	0.412**	—	0.307	-0.14	-0.236	—

Note: * $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$; — indicates insufficient data for correlation.

Fig. 3: Correlation between individual community smells and SATD types across all projects



conflicting relationships. Projects such as *P10* demonstrate both positive (OSE, BCE, RS, TF) and negative (SV, OS, UI) correlations within the same community, underscoring that a moderate increase in contributors can yield a mixture of organizational and communication patterns. In contrast, large projects show the highest frequency and diversity of significant results. Projects such as *P17* and *P20* display multiple strong correlations, both positive and negative. Notably, Unhealthy Interaction (UI) is more frequently associated with a negative correlation with SATD in large projects. Conversely, strong positive correlations with the Prima-donna Effect (PDE) and Toxic Communication (TC) are more prominent in large-scale settings, suggesting that issues such as power concentration and hostility become increasingly impactful as contributor communities grow.

Summary for RQ₂: Community smells consistently relate to SATD across ML projects, with RS, SV, and OSE being particularly prevalent. However, the influence of UI, PDE, and TC varies by project size and complexity.

C. Answering RQ₃: Correlation Between Community Smells and SATD Across ML Projects

We found that there are significant correlations between community smells and SATD instances across all the project sizes. In this section, we wanted to analyze how these community smells correlate with specific types of SATD. As shown in Fig. 3a, we observed that in small

projects, there exists strong and statistically significant positive correlations between specific community smells and particular types of SATD. Notably, Organization Silo Effect (OSE) shows strong correlations with multiple SATD categories, e.g., Code, Documentation, Testing, and Defect. This result implies that isolated communication structures are linked with both implementation and process-level debt. Similarly, Black Cloud Effect (BCE) and Truck Factor (TF) exhibit perfect correlations with Design debt and high correlations with Requirement debt, indicating that over-reliance on specific individuals contributes to architectural and design-related technical challenges. Conversely, smells like Sharing Villainy (SV) and Solution Defiance (SD) show moderate or weak associations across SATD types, while Radio Silence (RS) demonstrates a moderate link to Design and Requirement debt.

Compared to small projects, the associations are generally weaker but reveal meaningful patterns in Fig. 3b. Notably, Truck Factor (TF) exhibits the strongest and most consistent correlations across all SATD types, particularly with Defect, Documentation, and Requirement debt. Similarly, Organizational Silo Effect (OSE) and Radio Silence (RS) show moderate correlations with several SATD categories especially in Design, Defect and Requirement debt. In contrast, smells such as Unhealthy Interactions (UI) and Organizational Skirmish (OS) exhibit weak or even negative correlations across most SATD types. Notably, for large repositories, the results in Fig. 3c demonstrate strong and more widespread associations compared to small and medium-sized projects, which highlights how socio-technical challenges scale with

team size. Toxic Communications (TC) and Prima-donna Effect (PDE) exhibit particularly strong correlations across multiple SATD categories. TC shows the highest correlations with Code debt and notable associations with Documentation, Design, and Requirement debts. Similarly, PDE is strongly related to Code, Design, and Defect debt. Radio Silence (RS) also emerges as a prominent factor, moderately correlated with five SATD types, most notably Requirement and Test debt. Conversely, smells like Organizational Silo Effect (OSE) and Sharing Villainy (SV) show weaker or more isolated correlations.

Summary for RQ₃: In small projects, OSE, BCE, and TF are strongly linked to Code, Documentation, and Design debt. In medium projects, TF and RS exhibit moderate associations with Defect and Requirement debt. In large projects, TC and PDE emerge as dominant smells, correlating strongly with Code, Design, and Test debt.

D. Answering RQ₄: Trend Analysis Between Community Smells and SATD Across ML Projects

The Mann-Kendall trend analysis reveals evolving trajectories in both community smells and SATD types across ML projects, with notable differences based on project size. While RQ₃ reveals the strength and direction of associations, RQ₄ investigates the temporal dynamics between community smells and SATD across project releases. For instance, in small projects (P1–P8), we observe a clear presence of statistically significant increasing trends in several community smells such as Organization Silo Effect (OSE), Sharing Villainy (SV), and Radio Silence (RS), particularly in P1–P4. These findings suggest that communication breakdowns, isolated team structures, and uneven knowledge distribution tend to intensify over time, even in relatively small teams. Similarly, almost all SATD types, including Code, Documentation, and Requirement debt, show significant upward trends in most small projects, indicating early-stage accumulating debt.

Medium-sized projects (P9–P16) exhibit mixed trends in community smells and SATD. SV and OSE show strong upward trends, while SV and OS decline in P10, suggesting better collaboration. In P12, most smells show no trend except UI, which decreases significantly. Moreover, four types of SATD, i.e., Design, Documentation, Test, and Requirement debt, showed strong and statistically significant downward trends. However, code debt shows a significant increasing trend, implying that while design- and documentation-related debt is being addressed, implementation-level debt may still be accumulating.

In large-scale projects (P17–P22), Organizational Silo Effect (OSE) frequently trends upward, reflecting persistent structural fragmentation in large teams. The Black Cloud Effect (BCE) and Sharing Villainy (SV) also show a strong rising trend, suggesting growing communication gaps. Conversely, Unhealthy Interaction (UI) and Solution Defiance (SD) often

exhibit notable downward trends. However, Despite some improvements in socio-technical factors, all SATD types, except Design debt, continue to rise significantly, suggesting that technical debt persists even as collaboration issues partially improve.

Summary for RQ₄: Mann-Kendall trend analysis shows that small projects tend to accumulate both community smells (e.g., OSE, SV, RS) and all SATD types over time. Medium projects show mixed trends. However, in large projects, despite reductions in smells like UI and SD, most SATD types (except Design debt) consistently increase.

VI. KEY FINDINGS

Scaling Project, Swelling Debt. Prior work by Tamburri et al. [44] showed that community smells like OSE, PDE, and BCE are prevalent in open-source communities. Our findings from RQ₁ extend this to ML projects, where BCE, PDE, RS, TF, and TC consistently appear. In contrast, OSE and SV emerge mainly in larger teams, suggesting that subgroup formation and socio-technical debt increase with project size. This pattern aligns with Almarimi et al. [3], who linked turnover and communication gaps to community smell emergence. Further analysis of SATD distributions in Fig. 5 across project sizes verifies this observation. Our quartile analysis reveals that as ML projects grow in size, the presence and severity of SATD also increase. While small projects tend to accumulate a disproportionately high amount of requirement debt, medium and large projects experience sharp increases in defect, design, documentation, and test-related debt. Notably, large projects show high variability and upper quartiles across nearly all SATD types, suggesting that socio-technical debt manifests more broadly and intensively as teams scale. This finding strengthens the argument that the emergence of community smells is not only a symptom of poor social structure but also a potential precursor to SATD accumulation, particularly in complex ML ecosystems. Consequently, ML project leads should address community smell indicators and implement early and size-aware strategies to monitor and mitigate SATD growth over time.

Command, Conflict and Code Decay. Our findings from RQ₂ and RQ₃ empirically substantiate the theoretical claim articulated by Serebrenik [64], who hypothesized that specific community smells foster the introduction of SATD or reduce the likelihood of its elimination. Our project level analysis in RQ₂ shows that Radio Silence, Sharing Villainy and Organizational Silo Effect are the most reliable predictors of higher SATD counts across the ML projects which indicates their role as debt “originator”. Furthermore, our results from RQ₃ reveal that in larger projects, authority-centered smells like the Prima-donna Effect and interaction-based smells such as Toxic Communication are significantly correlated with persistent code, design, and maintenance debt. This correlation can be attributed to the socio-technical dynamics

Mann-Kendall Trend Summary by Project

OSE	72 [1]**	69 [1]**	32 [-]	112 [1]**	0 [-]	-6 [-]	20 [-]	-1 [-]	166 [1]**	8 [-]	216 [1]**	-115 [-]	15 [-]	144 [1]**	18 [1]**	0 [-]	98 [1]**	13 [-]	208 [1]**	186 [1]**	-14 [-]	141 [-]
BCE	-18 [-]	0 [-]	1 [-]	-15 [-]	5 [-]	0 [-]	0 [-]	-11 [-]	42 [-]	200 [1]**	0 [-]	62 [-]	-41 [-]	0 [-]	0 [-]	34 [-]	165 [1]**	151 [1]**	-160 [-]	188 [-]	-48 [-]	-28 [-]
PDE	0 [-]	0 [-]	0 [-]	0 [-]	0 [-]	0 [-]	0 [-]	0 [-]	0 [-]	0 [-]	0 [-]	0 [-]	0 [-]	0 [-]	0 [-]	0 [-]	185 [1]**	0 [-]	0 [-]	0 [-]	0 [-]	0 [-]
SV	90 [1]**	46 [1]**	60 [1]**	117 [1]**	-3 [-]	-6 [-]	90 [1]**	118 [-]	152 [1]**	-172 [1]**	146 [1]**	-207 [-]	56 [-]	380 [1]**	18 [1]**	-16 [-]	-70 [-]	-11 [-]	238 [1]**	-232 [1]**	0 [-]	-12 [-]
OS	0 [-]	0 [-]	0 [-]	0 [-]	0 [-]	0 [-]	0 [-]	0 [-]	0 [-]	-142 [1]**	0 [-]	-63 [-]	0 [-]	0 [-]	0 [-]	10 [-]	0 [-]	28 [-]	88 [-]	20 [-]	-48 [-]	116 [1]**
SD	24 [-]	44 [1]**	0 [-]	50 [-]	0 [-]	0 [-]	0 [-]	0 [-]	171 [-]	0 [-]	198 [1]**	-56 [-]	41 [-]	0 [-]	10 [-]	4 [-]	-62 [1]**	0 [-]	88 [-]	0 [-]	-4 [-]	-17 [-]
RS	-18 [-]	98 [1]**	0 [-]	137 [1]**	52 [1]**	0 [-]	-10 [-]	121 [-]	49 [-]	272 [1]**	242 [1]**	-88 [-]	46 [-]	-136 [-]	8 [-]	22 [-]	209 [1]**	73 [-]	12 [-]	358 [1]**	4 [-]	84 [-]
TF	-18 [-]	-23 [-]	0 [-]	0 [-]	23 [-]	0 [-]	0 [-]	0 [-]	77 [-]	158 [1]**	-19 [-]	63 [-]	0 [-]	0 [-]	8 [-]	4 [-]	48 [-]	118 [1]**	-152 [1]**	148 [-]	70 [-]	-43 [-]
UR	10 [-]	0 [-]	0 [-]	6 [-]	7 [-]	0 [-]	0 [-]	36 [-]	0 [-]	-38 [-]	0 [-]	-246 [1]**	41 [-]	0 [-]	-4 [-]	-26 [-]	-124 [1]**	-54 [-]	26 [-]	-232 [1]**	0 [-]	-35 [-]
TC	0 [-]	0 [-]	0 [-]	0 [-]	0 [-]	0 [-]	0 [-]	0 [-]	0 [-]	0 [-]	0 [-]	0 [-]	0 [-]	0 [-]	0 [-]	0 [-]	134 [1]**	27 [-]	0 [-]	0 [-]	0 [-]	0 [-]
Code	95 [1]**		340 [1]**		86 [1]**	162 [1]**	381 [1]**	660 [1]**	374 [1]**	1038 [1]**		1624 [1]**	0 [-]	-206 [-]	0 [-]	20 [-]	237 [1]**	-7 [-]	-58 [-]	1300 [1]**	-86 [-]	620 [1]**
Defect	1 [-]	109 [1]**	130 [1]**	212 [1]**	105 [1]**	0 [-]	116 [1]**	860 [1]**	445 [1]**	1008 [1]**		-111 [-]		2247 [1]**	41 [1]**	49 [-]	215 [1]**	23 [-]	943 [1]**	404 [1]**	-175 [-]	709 [1]**
Design	-18 [-]	0 [-]	97 [1]**	381 [1]**	62 [1]**	0 [-]	354 [1]**	3024 [1]**	446 [1]**	1045 [1]**		-637 [1]**		2671 [1]**	39 [1]**	138 [1]**	249 [1]**	-208 [1]**	19 [1]**	926 [1]**	280 [1]**	678 [1]**
Documentation	-142 [1]**		154 [1]**	0 [-]	90 [1]**		319 [1]**	927 [1]**	387 [1]**	1020 [1]**		1588 [1]**		1736 [1]**	32 [1]**	-18 [-]	354 [1]**	102 [-]	237 [1]**	1099 [1]**	0 [-]	566 [1]**
M&T	-140 [1]**		84 [1]**		56 [1]**	0 [-]	287 [1]**	952 [1]**	373 [1]**	808 [1]**		1062 [1]**		2540 [1]**	30 [1]**	40 [-]	-13 [-]	225 [1]**	779 [1]**	1031 [1]**	336 [1]**	515 [1]**
Requirement	-114 [1]**	183 [1]**	177 [1]**	263 [1]**	102 [1]**	168 [1]**	314 [1]**	996 [1]**	421 [1]**	1118 [1]**	448 [1]**	-916 [1]**		2393 [1]**	46 [1]**	99 [1]**	247 [1]**	302 [1]**	551 [1]**	1294 [1]**	658 [1]**	711 [1]**
	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	P11	P12	P13	P14	P15	P16	P17	P18	P19	P20	P21	P22

Fig. 4: Mann-Kendall trend analysis of community smells and SATD instances over releases

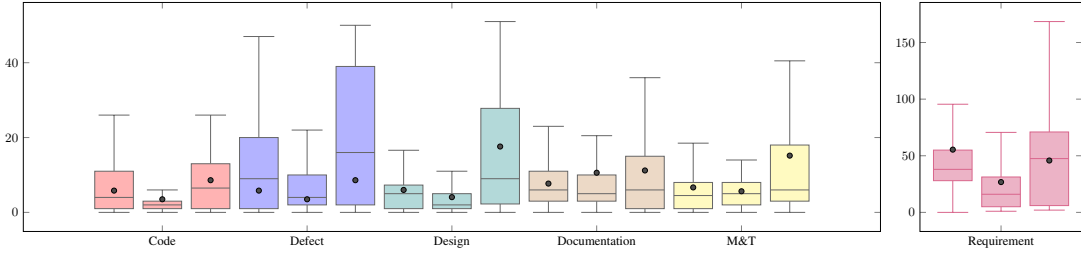


Fig. 5: Quartile distribution of SATD types by project size (small, medium, large)

TABLE VIII: Examples of community smells and SATD co-existence

Project	SATD Example	Observation
P8	<i>TODO PK do this once during construction so that we don't have to do it again,</i> Design Debt	This project had the SATD instance persisted from release 1.20 to 2.7, a period during BCE was constantly detected. Notably, BCE was no longer present in 2.7 and following this, the SATD was removed from the project. This temporal pattern suggest that resolving BCE-related socio-technical challenges, such as improved team communication and knowledge sharing enabled delayed design improvements.
P10	<i>todo why does it only work when we render a mesh before ?,</i> Defect Debt	This SATD instance persisted from release 1.1 to 2.2, during which the Truck Factor (TF) community smell was consistently detected. Following release 2.2, both the SATD and TF smell disappeared simultaneously. This pattern suggests that high TF can delay technical debt resolution, and that distributing team knowledge helps address lingering SATD, reinforcing the link between socio-technical bottlenecks and debt persistence.
P19	<i>TODO: xv and yv not used, should the next line be changed to xv, yv?,</i> Code Debt	We observed that this SATD instance persisted across releases 19.1 to 21.0, during which the Radio Silence (RS) community smell was also consistently detected. However, starting from release 22.0 and continuing through 23.0, RS was no longer present, and this SATD instance was also removed. This temporal alignment suggests that the resolution of RS potentially through improved communication and collaboration practices may have facilitated the identification and removal of that SATD.

prevalent in various development teams. In case of PDE, dominant individuals may centralize decision-making, resist collaborative input, and become bottlenecks for knowledge dissemination [1]. Such behavior can lead to architectural decisions that are not broadly evaluated, resulting in design choices that are difficult to modify or refactor. Similarly, TC creates a hostile communication environment, discouraging team members from engaging in open discussions or raising concerns about code quality [65]. This atmosphere can suppress proactive maintenance activities and the early identification of issues, allowing SATD to persist and grow. The findings from our study underscore the need for inclusive decision-making and communication platforms. Moreover, project managers should foster structured code reviews and establish psychological safety to encourage open dialogue,

mitigating TC's impact on the occurrence of SATD.

Additionally, we also attempted to find some examples of the co-existence of community smells and SATD from our studied projects. Table VIII illustrates some scenarios and our observations, which signify the potential interplay between socio-technical issues and the persistence of technical debt. These examples motivate the need for practitioner-centered validation to deepen the understanding of these dynamics.

Spikes Before the Storm, Reset at the Release. Our findings from **RQ₄** reveal the temporal evolution of community smells and SATD instances across project releases. Fig. 6 illustrates a clear similarity in the trends of community smells and SATD of the project *P12*, particularly around major releases. To better understand these trends, we analyzed the data, distinguishing between major and minor releases,

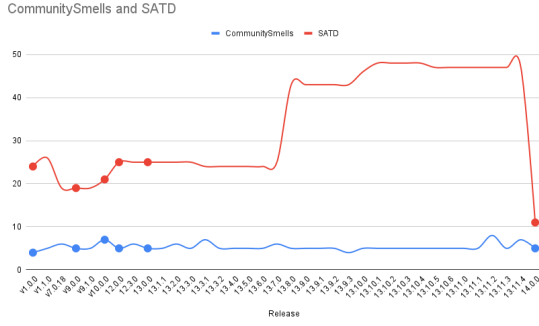


Fig. 6: Trend of community smells and SATD over major and minor releases in *P12*

classifying releases as major when adhering to the semantic versioning pattern *x.0.0* [66]. In project *P12*, we observed a notable combined upward and downward trend. Initially, both community smells and SATD showed relatively moderate and stable levels. However, around release 13.8.0, a significant increase occurred, particularly evident in SATD, which rose sharply and remained persistently high across subsequent minor releases. During this elevated SATD phase, there was also a noticeable increase in community smells, indicating concurrent socio-technical challenges related to collaboration and structural organization. Importantly, release 14.0.0 marked a clear inflection point, with both SATD and community smells sharply declining. This simultaneous reduction suggests that deliberate interventions, likely involving extensive refactoring, improved documentation, or structural adjustments, were undertaken. The temporal rise and fall of both metrics highlight their interconnection. Our findings from **RQ4** suggest that intensive development phases often fuel parallel growth in technical and social debt, while major releases offer key opportunities to reduce both and enhance project quality.

VII. THREATS TO VALIDITY

Threats to **construct validity** involve the alignment between theory and observation. Our study relies on the detection capabilities of automated tools, csDetector for community smells, and MT-BERT-SATD for identifying SATD. While both tools have demonstrated strong performance in prior work, they are inherently limited by their trained heuristics and model architectures. csDetector may miss smells occurring in off-platform or informal communication, and the SATD classifier may mislabel comments that do not follow conventional SATD phrasing. To address this, we conducted robustness checks and manual validation on a subset of projects to ensure the reliability of our findings (Section IV-B). Nonetheless, we acknowledge that not all community smells or SATD instances may be captured, and our results reflect patterns detectable by the tools. To maintain clarity, we explicitly refer to our findings as “community smells detectable by csDetector,” recognizing the tool’s scope and limitations.

Threats to **internal validity** involve factors within the study that may affect results. We aligned community smells and

SATD using release-based analysis, assuming they co-evolve within the same cycle. However, delays in their interaction may exist. The csDetector’s ML-based smell detection may introduce slight variability, which we mitigated through multiple runs and manual validation on a subset of projects.

Threats to **external validity** concern the generalizability of our findings. Our dataset, drawn from 155 active, high-quality ML projects curated by Latendresse et al. [43], strengthens the reliability of our results but may not fully represent immature, experimental, or closed-source projects. Additionally, as the projects are primarily from GitHub, they may not capture collaboration practices common in corporate environments. Moreover, while our dataset broadly targets ML systems, it does not isolate projects specific to deep learning (DL). As highlighted by Pepe et al. [67], DL systems often introduce unique forms of SATD such as unstable data pipelines, model retraining constraints, or hardware coupling, which are not explicitly addressed by general-purpose SATD classifiers like MT-BERT-SATD. As such, our SATD detection may underrepresent DL-specific debt instances, which presents a limitation in fully characterizing technical debt in deep learning ecosystems.

Threats to **conclusion validity** concern the soundness of statistical and logical inferences. We used Spearman’s rank and Point-Biserial correlation to examine links between community smells and SATD. While suitable for our data, these methods reveal co-occurrence, not causation. Temporal trends suggest alignment but not directionality. Future work should explore causal models to validate these patterns.

VIII. CONCLUSION AND FUTURE WORK

In this study, we conducted an empirical investigation into the relationship between community smells and SATD in 155 open-source ML-based software projects. Our analysis revealed that community smells are not only prevalent in ML systems but also closely associated with the accumulation and persistence of SATD. We identified specific smells, such as Radio Silence, Organizational Silo, and Sharing Villainy, as strong indicators of technical debt, particularly in requirements, design, and documentation. Furthermore, our release-level trend analysis shows that both community smells and SATD tend to escalate as projects scale, with large projects exhibiting high variability and persistence across most debt categories. These findings highlight the importance of tracking socio-technical indicators over time and emphasize the value of targeted interventions at major release points.

For future work, we plan to conduct qualitative studies involving developer interviews to better understand the contextual and cultural factors behind community smells. We also aim to explore the impact of community smell resolution on subsequent SATD reduction. Additionally, we hope to enhance detection accuracy by incorporating more advanced large language models and extending our analysis to industrial and non-GitHub ML projects, offering broader insights into socio-technical debt across diverse development environments.

IX. ACKNOWLEDGEMENT

This research is supported in part by the Natural Sciences and Engineering Research Council of Canada (NSERC) Discovery Grants program, the Canada Foundation for Innovation's John R. Evans Leaders Fund (CFI-JELF), and by the industry-stream NSERC CREATE in Software Analytics Research (SOAR).

REFERENCES

- [1] D. A. Tamburri, P. Kruchten, P. Lago, and H. v. Vliet, "Social debt in software engineering: insights from industry," *Journal of Internet Services and Applications*, vol. 6, pp. 1–17, 2015.
- [2] G. Catolino, F. Palomba, D. A. Tamburri, and A. Serebrenik, "Understanding community smells variability: A statistical approach," in *2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Society (ICSE-SEIS)*, 2021, pp. 77–86.
- [3] N. Almarimi, A. Ouni, and M. W. Mkaouer, "Learning to detect community smells in open source software projects," *Knowl. Based Syst.*, vol. 204, p. 106201, 2020. [Online]. Available: <https://api.semanticscholar.org/CorpusID:224991600>
- [4] D. A. Tamburri, F. Palomba, and R. Kazman, "Exploring community smells in open-source: An automated approach," *IEEE Transactions on Software Engineering*, vol. 47, no. 3, pp. 630–652, 2021.
- [5] N. Bettenburg and A. E. Hassan, "Studying the impact of social structures on software quality," in *2010 IEEE 18th International Conference on Program Comprehension*, 2010, pp. 124–133.
- [6] N. Nahar, S. Zhou, G. Lewis, and C. Kästner, "Collaboration challenges in building ml-enabled systems: communication, documentation, engineering, and process," in *Proceedings of the 44th International Conference on Software Engineering*, ser. ICSE '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 413–425. [Online]. Available: <https://doi.org/10.1145/3510003.3510209>
- [7] F. Zampetti, G. Fucci, A. Serebrenik, and M. Di Penta, "Self-admitted technical debt practices: a comparison between industry and open-source," *Empirical Software Engineering*, vol. 26, pp. 1–32, 2021.
- [8] A. Potdar and E. Shihab, "An exploratory study on self-admitted technical debt," in *2014 IEEE International Conference on Software Maintenance and Evolution*, 2014, pp. 91–100.
- [9] H. Sharp, H. Robinson, and M. Woodman, "Software engineering: Community and culture," *IEEE Softw.*, vol. 17, no. 1, p. 40–47, jan 2000. [Online]. Available: <https://doi.org/10.1109/52.819967>
- [10] G. Avelino, L. Passos, A. Hora, and M. T. Valente, "A novel approach for estimating truck factors," in *2016 IEEE 24th International Conference on Program Comprehension (ICPC)*. IEEE, 2016, pp. 1–10.
- [11] N. Raman, M. Cao, Y. Tsvetkov, C. Kästner, and B. Vasilescu, "Stress and burnout in open source: Toward finding, understanding, and mitigating unhealthy interactions," in *2020 IEEE/ACM 42nd International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER)*, 2020, pp. 57–60.
- [12] A. Murgia, M. Ortu, P. Tourani, B. Adams, and S. Demeyer, "An exploratory qualitative and quantitative analysis of emotions in issue report comments of open source systems," *Empirical Softw. Engg.*, vol. 23, no. 1, p. 521–564, feb 2018. [Online]. Available: <https://doi.org/10.1007/s10664-017-9526-0>
- [13] F. Palomba, D. A. Tamburri, F. A. Fontana, R. Oliveto, A. Zaidman, and A. Serebrenik, "Beyond technical aspects: How do community smells influence the intensity of code smells?" *IEEE transactions on software engineering*, vol. 47, no. 1, pp. 108–129, 2018.
- [14] F. Palomba, D. A. Tamburri, A. Serebrenik, A. Zaidman, F. A. Fontana, and R. Oliveto, "How do community smells influence code smells?" in *Proceedings of the 40th International Conference on Software Engineering: Companion Proceedings*, 2018, pp. 240–241.
- [15] D. Tamburri, R. Kazman, and W.-J. Van Den Heuvel, "Splicing community and software architecture smells in agile teams: An industrial study," 2019.
- [16] H. Mumtaz, P. Singh, and K. Blincoe, "Analyzing the relationship between community and design smells in open-source software projects: An empirical study," in *Proceedings of the 16th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, 2022, pp. 23–33.
- [17] A. Mailach and N. Siegmund, "Socio-technical anti-patterns in building ml-enabled software: Insights from leaders on the forefront," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 2023, pp. 690–702.
- [18] G. Annunziata, S. Lambiase, D. A. Tamburri, W.-J. van den Heuvel, F. Palomba, G. Catolino, F. Ferrucci, and A. De Lucia, "Uncovering community smells in machine learning-enabled systems: Causes, effects, and mitigation strategies," *ACM Transactions on Software Engineering and Methodology*, 2024.
- [19] "Replication package," 2025. [Online]. Available: <https://github.com/cynthia247/ESEM2025>
- [20] D. A. Tamburri, R. Kazman, and H. Fahimi, "The architect's role in community shepherding," *IEEE Software*, vol. 33, pp. 70–79, 2016. [Online]. Available: <https://api.semanticscholar.org/CorpusID:25842356>
- [21] N. Almarimi, A. Ouni, M. Chouchen, and M. W. Mkaouer, "Csdetector: An open source tool for community smells detection," in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2021. New York, NY, USA: Association for Computing Machinery, 2021, p. 1560–1564. [Online]. Available: <https://doi.org/10.1145/3468264.3473121>
- [22] W. Cunningham, "The wycash portfolio management system," in *Addendum to the Proceedings on Object-Oriented Programming Systems, Languages, and Applications (Addendum)*, ser. OOPSLA '92. New York, NY, USA: Association for Computing Machinery, 1992, p. 29–30. [Online]. Available: <https://doi.org/10.1145/157709.157715>
- [23] N. Brown, Y. Cai, Y. Guo, R. Kazman, M. Kim, P. Kruchten, E. Lim, A. MacCormack, R. Nord, I. Ozkaya, R. Sangwan, C. Seaman, K. Sullivan, and N. Zazworka, "Managing technical debt in software-reliant systems," in *Proceedings of the FSE/SDP Workshop on Future of Software Engineering Research*, ser. FoSER '10. New York, NY, USA: Association for Computing Machinery, 2010, p. 47–52. [Online]. Available: <https://doi.org/10.1145/1882362.1882373>
- [24] P. Kruchten, R. L. Nord, and I. Ozkaya, "Technical debt: From metaphor to theory and practice," *IEEE Software*, vol. 29, no. 6, pp. 18–21, 2012.
- [25] E. Lim, N. Taksande, and C. Seaman, "A balancing act: What software practitioners have to say about technical debt," *IEEE Software*, vol. 29, no. 6, pp. 22–27, 2012.
- [26] E. d. S. Maldonado and E. Shihab, "Detecting and quantifying different types of self-admitted technical debt," in *2015 IEEE 7th International Workshop on Managing Technical Debt (MTD)*, 2015, pp. 9–15.
- [27] S. Wehaibi, E. Shihab, and L. Guerrouj, "Examining the impact of self-admitted technical debt on software quality," in *2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, vol. 1, 2016, pp. 179–188.
- [28] F. Palomba, D. Andrew Tamburri, F. Arcelli Fontana, R. Oliveto, A. Zaidman, and A. Serebrenik, "Beyond technical aspects: How do community smells influence the intensity of code smells?" *IEEE Transactions on Software Engineering*, vol. 47, no. 1, pp. 108–129, 2021.
- [29] C. Sarmento, T. Massoni, A. Serebrenik, G. Catolino, D. Tamburri, and F. Palomba, "Gender diversity and community smells: A double-replication study on brazilian software teams," in *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 2022, pp. 273–283.
- [30] E. Caballero-Espinosa, J. C. Carver, and K. Stowers, "Community smells—the sources of social debt: A systematic literature review," *Inf. Softw. Technol.*, vol. 153, no. C, jan 2023. [Online]. Available: <https://doi.org/10.1016/j.infsof.2022.107078>
- [31] N. Almarimi, A. Ouni, M. Chouchen, and M. W. Mkaouer, "Improving the detection of community smells through socio-technical and sentiment analysis," *Journal of Software: Evolution and Process*, vol. 35, no. 6, p. e2505, 2023.
- [32] N. Almarimi, A. Ouni, M. Chouchen, I. Saidani, and M. W. Mkaouer, "On the detection of community smells using genetic programming-based ensemble classifier chain," in *Proceedings of the 15th International Conference on Global Software Engineering*, ser. ICGSE '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 43–54. [Online]. Available: <https://doi.org/10.1145/3372787.3390439>
- [33] G. Voria, V. Pentangelo, A. D. Porta, S. Lambiase, G. Catolino, F. Palomba, and F. Ferrucci, "Community smell detection and refactoring in slack: The cadocs project," in *2022 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2022, pp. 469–473.
- [34] S. Brandstädter and K. Sonntag, "Interdisciplinary collaboration: How to foster the dialogue across disciplinary borders?" in *Advances in*

Ergonomic Design of Systems, Products and Processes: Proceedings of the Annual Meeting of GfA 2015. Springer, 2016, pp. 395–409.

- [35] G. F. C. Brown, *Factors that facilitate or inhibit interdisciplinary collaboration within a professional bureaucracy*. University of Arkansas, 1995.
- [36] S. Passi and P. Sengers, “Making data science systems work,” *Big data & society*, vol. 7, no. 2, p. 2053951720939605, 2020.
- [37] G. Sierra, E. Shihab, and Y. Kamei, “A survey of self-admitted technical debt,” *Journal of Systems and Software*, vol. 152, pp. 70–82, 2019. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0164121219300457>
- [38] B. Russo, M. Camilli, and M. Mock, “Weaksatd: Detecting weak self-admitted technical debt,” in *Proceedings of the 19th International Conference on Mining Software Repositories*, ser. MSR ’22. New York, NY, USA: Association for Computing Machinery, 2022, p. 448–453. [Online]. Available: <https://doi.org/10.1145/3524842.3528469>
- [39] R. Maipradit, C. Treude, H. Hata, and K. Matsumoto, “Wait for it: identifying “on-hold” self-admitted technical debt,” *Empirical Software Engineering*, vol. 25, pp. 3770–3798, 2020.
- [40] M. A. de Freitas Farias, M. G. de Mendonça Neto, M. Kalinowski, and R. O. Spínola, “Identifying self-admitted technical debt through code comment analysis with a contextualized vocabulary,” *Information and Software Technology*, vol. 121, p. 106270, 2020.
- [41] D. OBrien, S. Biswas, S. Imtiaz, R. Abdalkareem, E. Shihab, and H. Rajan, “23 shades of self-admitted technical debt: An empirical study on machine learning software,” in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2022. New York, NY, USA: Association for Computing Machinery, 2022, p. 734–746. [Online]. Available: <https://doi.org/10.1145/3540250.3549088>
- [42] A. Bhatia, F. Khomh, B. Adams, and A. E. Hassan, “An empirical study of self-admitted technical debt in machine learning software,” 2023.
- [43] J. Latendresse, S. Abedu, A. Abdellatif, and E. Shihab, “An exploratory study on machine learning model management,” *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 1, pp. 1–31, 2024.
- [44] D. A. Tamburri, F. Palomba, and R. Kazman, “Exploring community smells in open-source: An automated approach,” *IEEE Transactions on software Engineering*, vol. 47, no. 3, pp. 630–652, 2019.
- [45] D. OBrien, S. Biswas, S. Imtiaz, R. Abdalkareem, E. Shihab, and H. Rajan, “23 shades of self-admitted technical debt: an empirical study on machine learning software,” in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2022, pp. 734–746.
- [46] N. Almarimi, A. Ouni, M. Chouchen, and M. W. Mkaouer, “csdetector: an open source tool for community smells detection,” in *ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2021, pp. 1560–1564.
- [47] T. Neugebauer, P. Lasou, and P. Carson, “What are the characteristic community smells influencing the sustainability of open-source repository software communities?” 2024.
- [48] E. Caballero-Espinosa, J. C. Carver, and K. Stowers, “Community smells—the sources of social debt: A systematic literature review,” *Information and Software Technology*, vol. 153, p. 107078, 2023.
- [49] J. Linåker, E. Papatheocharous, and T. Olsson, “How to characterize the health of an open source software project? a snowball literature review of an emerging practice,” in *Proceedings of the 18th International Symposium on Open Collaboration*, 2022, pp. 1–12.
- [50] E. Jabrayilzade, M. Evtikhiev, E. Tüzün, and V. Kovalenko, “Bus factor in practice,” in *Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice*, 2022, pp. 97–106.
- [51] D. A. Tamburri, F. Palomba, A. Serebrenik, and A. Zaidman, “Discovering community patterns in open-source: a systematic approach and its evaluation,” *Empirical Software Engineering*, vol. 24, pp. 1369–1417, 2019.
- [52] G. Voria, V. Pentangelo, A. Della Porta, S. Lambiase, G. Catolino, F. Palomba, and F. Ferrucci, “Community smell detection and refactoring in slack: The cadocs project,” in *2022 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2022, pp. 469–473.
- [53] A. Della Porta, S. Lambiase, G. Catolino, F. Ferrucci, and F. Palomba, “A novel, tool-supported catalog of community smell symptoms,” 2024.
- [54] J. M. Faria, “Non-determinism and failure modes in machine learning,” in *2017 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*. IEEE, 2017, pp. 310–316.
- [55] J. H. Zar, “Spearman rank correlation,” *Encyclopedia of biostatistics*, vol. 7, 2005.
- [56] S. Prion and K. A. Haerling, “Making sense of methods and measurement: Spearman-rho ranked-order correlation coefficient,” *Clinical Simulation in Nursing*, vol. 10, no. 10, pp. 535–536, 2014.
- [57] M. A. Bujang and N. Baharum, “Sample size guideline for correlation analysis,” *World*, vol. 3, no. 1, pp. 37–46, 2016.
- [58] V. Pontillo, D. Amoroso d’Aragona, F. Pecorelli, D. Di Nucci, F. Ferrucci, and F. Palomba, “Machine learning-based test smell detection,” *Empirical Software Engineering*, vol. 29, no. 2, p. 55, 2024.
- [59] H. Gu, S. Zhang, Q. Huang, Z. Liao, J. Liu, and D. Lo, “Self-admitted technical debts identification: How far are we?” in *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 2024, pp. 804–815.
- [60] M. S. Sheikhaei, Y. Tian, S. Wang, and B. Xu, “An empirical study on the effectiveness of large language models for satd identification and classification,” *Empirical Software Engineering*, vol. 29, no. 6, p. 159, 2024.
- [61] D. Kornbrot, “Point biserial correlation,” *Wiley StatsRef: Statistics Reference Online*, 2014.
- [62] R. F. Tate, “Correlation between a discrete and a continuous variable. point-biserial correlation,” *The Annals of mathematical statistics*, vol. 25, no. 3, pp. 603–607, 1954.
- [63] K. H. Hamed and A. R. Rao, “A modified mann-kendall trend test for autocorrelated data,” *Journal of hydrology*, vol. 204, no. 1-4, pp. 182–196, 1998.
- [64] A. Serebrenik, “Social software engineering,” 2022.
- [65] J. Sayago-Heredia, G. C. Sailema, R. Pérez-Castillo, and M. Piattini, “Analyzing the correlation between toxic comments and code quality,” *Journal of Software: Evolution and Process*, vol. 37, no. 2, p. e2739, 2025.
- [66] J. Wu, W. Xu, K. Gao, J. Li, and M. Zhou, “Characterize software release notes of github projects: structure, writing style, and content,” in *2023 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 2023, pp. 473–484.
- [67] F. Pepe, F. Zampetti, A. Mastropaolo, G. Bavota, and M. Di Penta, “A taxonomy of self-admitted technical debt in deep learning systems,” in *2024 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2024, pp. 388–399.